

Pipeline de Dados com IoT e Docker

Processamento, armazenamento e visualização de leituras de temperatura de dispositivos IoT

Disciplina: Disruptive Architectures — IoT, Big Data e IA

Autora: Thais Ribeiro de Aquino

Curso: Análise e Desenvolvimento de Sistemas — UniFECAF

1. Contextualização do projeto

Dispositivos IoT (Internet of Things) geram grandes volumes de dados em tempo real — temperatura, umidade, presença, consumo de energia — que precisam ser coletados, armazenados e analisados de forma estruturada para gerar valor. Este projeto propõe um pipeline de dados completo, do arquivo bruto ao dashboard interativo, para leituras de temperatura coletadas por sensores IoT.

O conjunto de dados utilizado é o **Temperature Readings: IoT Devices**, disponível publicamente no Kaggle, contendo 97.606 leituras de temperatura (em °C) de um sensor instalado em uma sala, classificadas como internas ("In") ou externas ("Out"), coletadas entre julho e dezembro de 2018.

O objetivo é construir um pipeline que: (1) processe o CSV bruto em Python; (2) armazene os dados de forma estruturada em um banco de dados relacional PostgreSQL, executado em um container Docker; (3) disponibilize views SQL que resumam os dados para análise; e (4) exiba essas análises em um dashboard interativo construído com Streamlit e Plotly, permitindo interpretar rapidamente o comportamento térmico do ambiente monitorado.

2. Descrição dos passos realizados

2.1 Configuração do ambiente

Instalação do Python 3, Docker e criação de um ambiente virtual Python (venv). Instalação das dependências do projeto via `pip install -r requirements.txt` (pandas, psycopg2-binary, SQLAlchemy, streamlit, plotly).

2.2 Criação do container PostgreSQL

O banco de dados PostgreSQL 16 é executado em um container Docker, definido em `docker-compose.yml` (equivalente ao comando `docker run --name postgres-iot -e POSTGRES_PASSWORD=... -p 5432:5432 -d postgres` pedido no enunciado), expondo a porta 5432 e persistindo os dados em um volume Docker.

2.3 Desenvolvimento do ETL em Python

O script `src/etl.py` implementa as três etapas clássicas de um pipeline de dados: **Extract** (leitura do CSV com pandas), **Transform** (renomeação de colunas, remoção de 1 linha duplicada, conversão de datas e criação da coluna `device_id`) e **Load** (criação da tabela `temperature_readings` e inserção dos 97.605 registros válidos via SQLAlchemy).

2.4 Criação das views SQL

Após a carga dos dados, o próprio script executa o arquivo `sql/views.sql`, que cria quatro views de análise (detalhadas na seção 3) diretamente no banco PostgreSQL.

2.5 Construção do dashboard

O dashboard interativo (`src/dashboard.py`) usa Streamlit para a interface web e Plotly para os gráficos, lendo os dados diretamente das views SQL via SQLAlchemy/pandas (`pd.read_sql`).

3. Explicação detalhada das views SQL

Foram criadas quatro views (três obrigatórias mais uma bônus), todas construídas sobre a tabela `temperature_readings`:

avg_temp_por_dispositivo

Agrupar as leituras por dispositivo (sensor interno x externo) e calcular temperatura média, mínima, máxima e o total de leituras de cada um. Permite comparar rapidamente o comportamento térmico de cada sensor.

leituras_por_hora

Extraí a hora do dia (0–23) de cada leitura e conto quantas leituras ocorreram em cada hora, somando todos os dias do período. Revela em quais horários há mais atividade de amostragem dos sensores.

temp_max_min_por_dia

Agrupar as leituras por dia e calcular a temperatura máxima, mínima e média daquele dia. Permite acompanhar a variação térmica diária ao longo de todo o período coletado.

temp_media_por_hora_dispositivo (bônus)

Combina as duas dimensões anteriores: temperatura média por hora do dia, separada por dispositivo. Evidencia o ciclo diário de aquecimento/resfriamento de cada sensor.

Trecho de exemplo (view completa em `sql/views.sql`):

```
CREATE VIEW avg_temp_por_dispositivo AS
SELECT device_id, ROUND(AVG(temp),2) AS avg_temp, MIN(temp) AS min_temp,
       MAX(temp) AS max_temp, COUNT(*) AS total_leituras
FROM temperature_readings
GROUP BY device_id;
```

4. Visualizações geradas no dashboard

As imagens abaixo foram capturadas a partir da execução real do dashboard (`streamlit run src/dashboard.py`), já conectado ao PostgreSQL com os dados carregados pelo pipeline.

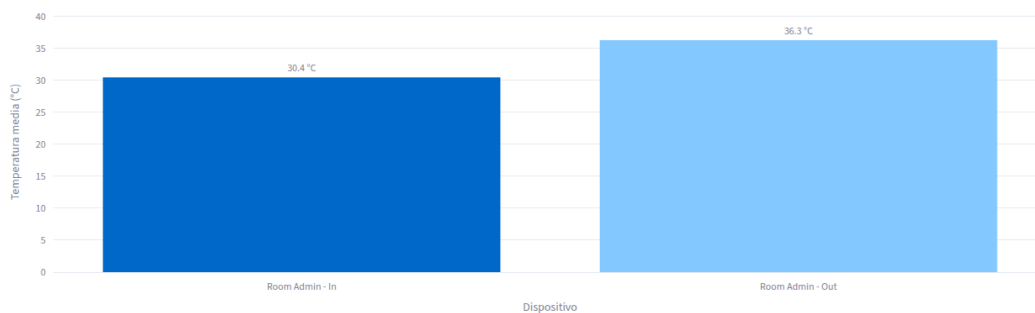


Gráfico 1 — Média de temperatura por dispositivo (interno x externo).

Leituras por Hora do Dia

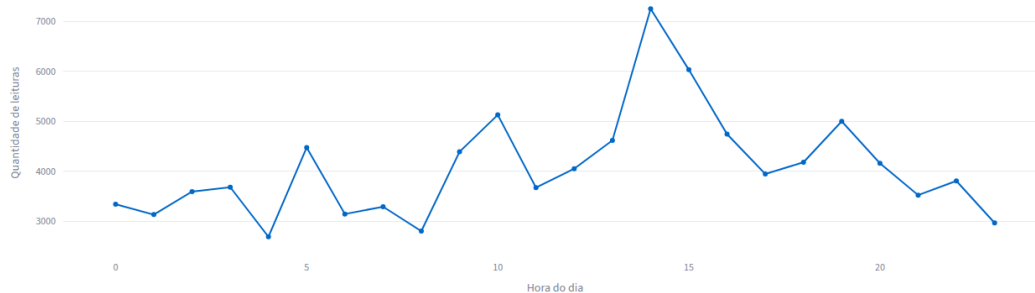
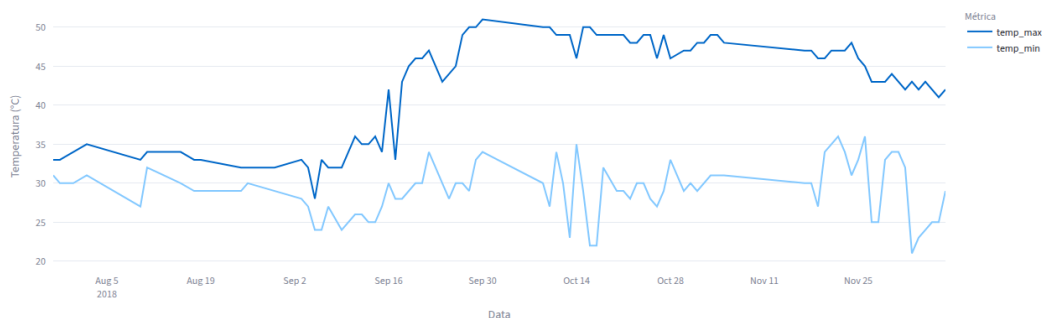


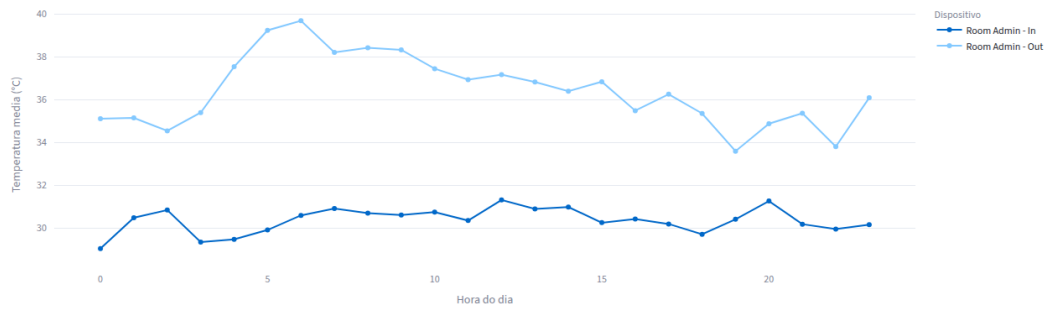
Gráfico 2 — Contagem de leituras por hora do dia.

Temperaturas Máximas e Mínimas por Dia



Ciclo Diário de Temperatura por Dispositivo

Gráfico 3 — Temperaturas máximas e mínimas por dia.



Pipeline de Dados com IoT e Docker — Disruptive Architectures: IoT, Big Data e IA

Gráfico 4 (bônus) — Ciclo diário de temperatura por dispositivo.

5. Principais insights obtidos

- O sensor externo ("Out") registra, em média, cerca de 5,9 °C a mais que o sensor interno ("In") — 36,3 °C contra 30,4 °C — coerente com um ambiente interno climatizado/protegido em relação à variação externa.
- A amplitude térmica do sensor externo é bem maior (24 °C a 51 °C, 27 °C de amplitude) do que a do sensor interno (21 °C a 41 °C, 20 °C de amplitude): o ambiente interno amortece picos de calor.
- O ciclo diário do sensor externo é muito mais pronunciado: a temperatura média sobe de cerca de 35 °C de madrugada para quase 40 °C entre 5h e 8h, enquanto o sensor interno oscila pouco (29 °C a 31 °C) ao longo de todo o dia.
- A frequência de leituras não é uniforme: há um pico claro por volta das 14h, com mais de 7.200 leituras somadas no período — mais que o dobro da média das demais horas.
- A partir de meados de setembro de 2018, as temperaturas máximas diárias sobem visivelmente (de cerca de 35 °C para 45–50 °C), indicando uma provável mudança sazonal ou de posicionamento do sensor externo.

Sugestões de uso prático em um ambiente real

- Disparar alertas automáticos quando a temperatura interna ultrapassar uma faixa segura (por exemplo, acima de 32–35 °C), indicando possível falha de climatização.
- Usar a comparação interno x externo para otimizar climatização e consumo de energia (ex.: pré-resfriar o ambiente antes dos horários de pico de temperatura externa).
- Detectar anomalias de sensor (leituras muito fora do padrão histórico por hora/dia) como indício de defeito de hardware.
- Estender o pipeline para múltiplos dispositivos e salas reais, já que a estrutura de `device_id` no banco de dados já suporta esse cenário sem alterações no modelo.

Repositório do projeto (código-fonte completo, views SQL, Docker e README):
<https://github.com/ThaisRAquino/pipeline-dados-iot-docker>